



Procesadores de lenguajes

Ingeniería Informática
Especialidad de Computación
Tercer curso, segundo cuatrimestre



Escuela Politécnica Superior de Córdoba
Universidad de Córdoba

Curso académico: 2025 - 2026

TRABAJO DE PREPARACIÓN DEL EXAMEN DE PRÁCTICAS

1. Introducción

- **Competencias**
 - El presente trabajo de prácticas pretende desarrollar las siguientes “competencias de la asignatura”:
 - CU1. Acreditar el uso y dominio de una lengua extranjera.
 - CTEC2. Capacidad para **conocer** los fundamentos teóricos de los lenguajes de programación y las técnicas de procesamiento léxico, sintáctico y semántico asociadas, y saber **aplicarlas para la creación, diseño y procesamiento de lenguajes**.
- **Objetivo**
 - Se debe utilizar **flex** y **bison** para elaborar un intérprete de código
 - *interpreter.exe*
- **Importante**
 - Este trabajo se debe elaborar a partir del **ejemplo 17** explicado en clase de prácticas.

2. Elaboración y entrega del trabajo

- **Modo de realización del trabajo**
 - El trabajo se podrá realizar de forma individual o por parejas.
- **Plazo de entrega**
 - Hasta las 8:30 horas del martes 26 de mayo de 2026.
- **Modo de entrega**
 - Un fichero comprimido deberá ser “subido” a la tarea de la plataforma de “*moodle*”.
 - Dicho fichero comprimido deberá contener:
 - Fichero de flex
 - Fichero de bison
 - Ficheros de C++ (“*.cpp*”, “*.hpp*”)
 - Fichero makefile
 - Ficheros de ejemplo de pseudocódigo con la extensión “*.p*”

3. Características de lenguaje de pseudocódigo

a) Componentes léxicos o *tokens*

- **Palabras reservadas**
 - **Sentencias de control**
 - ✓ *read, read_string*
 - ✓ *print*
 - ✓ *if, then, else, end_if*
 - ✓ *while, do, end_while*
 - ✓ *repeat, until, for, end_for, from, step, to*
 - ✓ *switch, case, default, end_switch*
 - **Manejo de la pantalla**
 - ✓ *clear_screen, place*
 - **Funciones predefinidas**
 - ✓ *sin, cos, sqrt, log, log10, exp, sqrt, integer, abs.*
 - **Constantes predefinidas**
 - ✓ *pi, e, gamma, phi, deg*
 - **Operador matemático**
 - ✓ *mod*
 - **Constantes y operadores lógicos**
 - ✓ *true, false*
 - ✓ *or, and, not*
 - **Observaciones**
 - ✓ No se distinguirá entre mayúsculas ni minúsculas.
 - ✓ Las palabras reservadas no se podrán utilizar como identificadores.
- **Identificadores**
 - **Características**
 - ✓ Estarán compuestos por una serie de letras, dígitos y el símbolo de subrayado “_”.
 - ✓ Deben comenzar por una letra
 - ✓ No podrán acabar con el símbolo de subrayado, ni tener dos subrayados seguidos.
 - **Identificadores válidos:**
 - ✓ *dato, dato_1, dato_1_a*
 - **Identificadores no válidos:**
 - ✓ *_dato, dato_, dato__1*
 - No se distinguirá entre mayúsculas ni minúsculas.
- **Número**
 - Se utilizarán números enteros, reales de punto fijo y reales **con notación científica**.
 - Todos ellos serán tratados conjuntamente como números.
- **Cadena**
 - Estará compuesta por una serie de caracteres delimitados por comillas simples:
 - ✓ *‘Ejemplo de cadena’*
 - ✓ *‘Ejemplo de cadena con salto de línea \n y tabulador \t’*

- Deberá permitir la inclusión de la comilla simple utilizando la barra (\):
 - ✓ *‘Ejemplo de cadena con \' comillas\' simples’.*
- **Notas:**
 - ✓ Las comillas exteriores no se almacenarán como parte de la cadena.
 - ✓ Por tanto, al escribir una cadena,
 - no se deberán imprimir las comillas exteriores
 - pero sí se deberán interpretar los comandos \n, \t, \\, y \’.
- **Operador de asignación**
 - asignación: :=
- **Operadores aritméticos**
 - suma: +
 - ✓ Unario: + 2
 - ✓ Binario: 2 + 3
 - resta: -
 - ✓ Unario: - 2
 - ✓ Binario: 2 - 3
 - producto: *
 - división: /
 - división entera: //
 - módulo: *mod*
 - potencia: ^
- **Operador alfanumérico:**
 - concatenación: ||
- **Operadores relacionales de números y cadenas:**
 - menor que: <
 - menor o igual que: <=
 - mayor que: >
 - mayor o igual: >=
 - igual que: =
 - distinto que: <>
 - Por ejemplo:
 - ✓ si *A* es una variable numérica y *control* una variable alfanumérica, se pueden generar las siguientes expresiones relacionales:

$(A \geq 0)$
 $(control \neq 'stop')$

 $valor := (a > b)$
 $if (valor = true) then \dots end_if$
 $if (valor \neq false) then \dots end_if$
- **Operadores lógicos**
 - disyunción lógica: *or*
 - conjunción lógica: *and*
 - negación lógica: *not*
 - ✓ Por ejemplo:

$(A \geq 0) \text{ and } not (control \neq 'stop')$

- **Comentarios**
 - De bloque o de varias línea.
 - ✓ Delimitados por los símbolos *(* y *)*
*(* ejemplo de comentario de varias líneas *)*
 - ✓ Se deben evitar los comentarios de bloque anidados.
 - De una línea
 - ✓ Todo lo que siga al carácter *#* hasta el final de la línea.
ejemplo de comentario de una línea
 - ✓ Nota:
 - Se debe eliminar la opción de terminar un programa cuando el símbolo “#” aparece al principio de una línea.
- **Punto y coma: “;”**
 - Se utilizará para indicar el fin de una sentencia.

b) Sentencias

- **Asignación**
 - ***identificador := expresión numérica***
 - ✓ Declara a ***identificador*** como una variable numérica y le asigna el valor de la expresión numérica.
 - ✓ Las expresiones numéricas se formarán con números, variables numéricas y operadores numéricos.
 - ***identificador := expresión alfanumérica***
 - ✓ Declara a ***identificador*** como una variable alfanumérica y le asigna el valor de la expresión alfanumérica.
 - ✓ Las expresiones alfanuméricas se formarán con cadenas, variables alfanuméricas y el operador alfanumérico de concatenación (*||*).
- **Lectura**
 - ***read (identificador)***
 - ✓ Declara a ***identificador*** como variable numérica y le asigna el número leído.
 - ***read_string (identificador)***
 - ✓ Declara a ***identificador*** como variable alfanumérica y le asigna la cadena leída (sin comillas).
- **Escritura**
 - ***print (expresión numérica)***
 - ✓ El valor de la expresión numérica es escrito en la pantalla.
 - ***print (expresión lógica)***
 - ✓ El valor de la expresión lógica es escrito en la pantalla.
 - ***print (expresión alfanumérica)***
 - ✓ La cadena (sin comillas exteriores) es escrita en la pantalla.

- ✓ Se debe permitir la interpretación de comandos de saltos de línea (\n) y tabuladores (\t) que puedan aparecer en la expresión alfanumérica.

```
print('\t Introduzca el dato \n');  
Introduzca el dato
```

- **Sentencias de control**¹

- Sentencia *condicional* simple

```
if condición  
    then lista de sentencias  
end_if
```

- Sentencia *condicional* compuesta

```
if condición  
    then lista de sentencias  
    else lista de sentencias  
end_if
```

- Bucle “*while*”

```
while condición do  
    lista de sentencias  
end_while
```

- Bucle “*repeat*”

```
repeat  
    lista de sentencias  
until condición
```

- Bucle “*for*”

```
for identificador  
    from expresión numérica 1  
    to expresión numérica 2  
    [step expresión numérica 3]  
    do  
        lista de sentencias  
end_for
```

- ✓ Notas

- El *paso* (*step*) es opcional; en su defecto, tomará el valor 1.
- Se debe controlar que el bucle esté bien definido, es decir, que el intervalo de valores del identificador no sea nulo o infinito.

- Sentencia “*switch*”

```
switch (expresión)  
    case expresión 1: lista de sentencias  
    case expresión 2: lista de sentencias  
    ...  
    [default: lista de sentencias]  
end_switch
```

¹ Una condición será una expresión relacional o una expresión lógica compuesta.

- **Comandos especiales**
 - ***clear_screen***
 - ✓ borra la *pantalla*
 - ***place*(expresión numérica1, expresión numérica2)**
 - ✓ Coloca el cursor de la pantalla en las coordenadas indicadas por los valores de las expresiones numéricas.
- **Observación**
 - Se debe permitir que una variable pueda cambiar de tipo durante la ejecución del intérprete.
 - ✓ Ejemplo

```
# La variable dato es numérica
dato := 10;
print(dato);
...
# La variable dato se convierte en alfanumérica
read_string(dato);
print(dato);
```
- **Se valorará la ampliación del lenguaje de pseudocódigo**
 - **Ejemplos**
 - ✓ Sentencia ***do { ... } while***
 - ✓ Operador alternativa “?”
 - $a := (b \geq 0)? b, -b;$
 - ✓ Operadores unarios: ++, --, ! (factorial), etc.
 - ✓ Nuevas funciones matemáticas: factorial, etc.
 - ✓ Operadores aritméticos y de asignación: +=, -=, etc.
 - ✓ Manejo de pantalla: negrita, color, etc.
 - ✓ Etc.

4. Control de errores

El intérprete deberá controlar toda clase de errores:

- **Léxicos:**
 - Identificador mal escrito.
 - Un número mal escrito.
 - Operadores mal escritos.
 - Utilización de símbolos no permitidos.
 - Comentarios de bloque anidados.
 - Etc.
- **Sintácticos:**
 - Sentencias de control más escritas.
 - Sentencias con argumentos incompatibles.
 - Etc.
 - **Observación**
 - Se valorará la utilización de “reglas de producción de control de errores” que **no** generen conflictos.
- **Semánticos**
 - Argumentos u operandos incompatibles.
- **De ejecución**
 - Sentencia “*para*” que pueda generar un bucle infinito.
 - Fichero de entrada inexistente o con una extensión incorrecta.
 - Etc.

5. Modos de ejecución del intérprete

El intérprete se podrá ejecutar de dos formas diferentes:

- **Modo interactivo**
 - Se ejecutarán las instrucciones tecleadas desde un terminal de texto

```
interpreter.exe
> ...
```
 - Se utilizará el carácter de fin de fichero para terminar la ejecución:
 - Control + D
- **Ejecución desde un fichero**
 - Se interpretarán las sentencias de un fichero pasado como argumento desde la línea de comandos
 - El fichero deberá tener la extensión “.p”

```
interpreter.exe example.p
```

6. Criterios de evaluación

- Se utilizará la siguiente **Tabla de Evaluación**

	Necesita mejorar	Puede mejorar	Aceptable	Bien	Muy bien
Gramática sin conflictos					
Ejecución del intérprete					
Interactiva					
A partir de un fichero					
Complejidad del lenguaje					
Componentes léxicos					
Sentencias					
Control de errores					
Errores léxicos					
Errores sintácticos					
Errores semánticos					
Errores de ejecución					
Ampliación del lenguaje					
Ejemplos					
Cantidad					
Originalidad					
Complejidad					
Documentación con doxygen					
Asistencia a clase de prácticas					

- Observaciones
 - La gramática **no** deberá tener ningún conflicto.
 - Esta condición es **imprescindible** para aprobar el trabajo de prácticas.
 - El intérprete deberá funcionar correctamente en “ThinStation” de la Universidad de Córdoba.
 - En particular, deberán interpretar correctamente los ejemplos proporcionados por el profesor
 - menu.p, binario.p, conversion.p, if.p, for.p, test_switch.p
 - Se deberán proponer nuevos ejemplos que muestren el uso de las sentencias y operadores del lenguaje de pseudocódigo:
 - read, print,*
 - read_string*
 - if, while, repeat, for, switch,*
 - ...
 - Se valorará la cantidad, originalidad y complejidad de los ejemplos propuestos.